

15 MINI PROJECTS — Complete by End of Day 5

Each mini project is designed to be completed within 2-4 hours, using concepts taught on specific days. Teams of 2-3 students. All projects must be in a GitHub repo with a working CI pipeline and Dockerized deployment.

MINI PROJECT #01: Linux Sysadmin Automation Kit [Best done on: Day 1]

Project Brief: Write a bash script that performs automated server health checks — checks disk space, RAM usage, running services, and sends a formatted report to a log file. Add a cron job to run it every hour.

Skills Applied: Linux, Bash, Cron, Logging

Success Outcome: *A deployable monitoring script that a sysadmin would actually use.*

MINI PROJECT #02: Git Workflow Simulator [Best done on: Day 2]

Project Brief: Create a shared GitHub repository with branch protection rules. Simulate a 3-person team workflow: feature branches, PRs, code reviews, merge conflicts, and release tagging. Document the workflow in a CONTRIBUTING.md.

Skills Applied: Git, GitHub, Branch Strategy, Docs

Success Outcome: *A fully configured team repo with documented workflow — show how professional teams collaborate.*

MINI PROJECT #03: Dockerized To-Do API [Best done on: Day 2-3]

Project Brief: Build a simple REST API (Python Flask or Node.js) with CRUD operations. Write a multi-stage Dockerfile to minimize image size. Push to Docker Hub with semantic versioning tags.

Skills Applied: Docker, Dockerfile, REST API, Docker Hub

Success Outcome: *A containerized API available publicly on Docker Hub.*

MINI PROJECT #04: CI Pipeline for a Python App [Best done on: Day 3]

Project Brief: Take any small Python script, write pytest unit tests for it, and set up a GitHub Actions workflow that runs on every push: linting (flake8), unit tests, and coverage report. Badge the README with build status.

Skills Applied: GitHub Actions, pytest, Python, CI

Success Outcome: *A repo with a green build badge and automated quality gates.*

MINI PROJECT #05: Docker Compose Full-Stack App [Best done on: Day 2-3]

Project Brief: Deploy a 3-container application: a web frontend (Nginx), a backend API (Flask), and a database (PostgreSQL). All defined in a single docker-compose.yml. Data persists across container restarts.

Skills Applied: Docker Compose, Nginx, PostgreSQL, Networking

Success Outcome: *A multi-tier app running with one command: docker-compose up.*

MINI PROJECT #06: Automated Artifact Versioning Pipeline [Best done on: Day 3]

Project Brief: Create a GitHub Actions pipeline that automatically builds a Docker image, tags it with the git commit SHA and semantic version, pushes to Docker Hub, and creates a GitHub Release with changelog.

Skills Applied: GitHub Actions, Docker, Semantic Versioning, Release Automation

Success Outcome: *Every git tag creates a versioned, deployable release automatically.*

MINI PROJECT #07: Kubernetes Self-Healing App Demo [Best done on: Day 4]

Project Brief: Deploy a web application to minikube with a Deployment (3 replicas), a ClusterIP Service, and a NodePort for external access. Write a shell script that deletes pods randomly and demonstrates auto-healing in a video/GIF.

Skills Applied: Kubernetes, kubectl, YAML, minikube

Success Outcome: *Visual proof of K8s self-healing — great for portfolio/LinkedIn.*

MINI PROJECT #08: Kubernetes Horizontal Pod Autoscaler [Best done on: Day 4]

Project Brief: Deploy a CPU-intensive app to Kubernetes. Configure HPA to scale from 2 to 8 pods based on CPU usage. Use the hey load testing tool to generate traffic and capture a screen recording of pods scaling up and down.

Skills Applied: Kubernetes, HPA, Load Testing, Metrics Server

Success Outcome: *Demonstrated auto-scaling under load — a core DevOps capability.*

MINI PROJECT #09: Prometheus + Grafana Monitoring Stack [Best done on: Day 4]

Project Brief: Deploy Prometheus and Grafana to minikube via Helm charts. Configure Prometheus to scrape a sample app. Build a Grafana dashboard showing request rate, error rate, and pod CPU/memory. Export dashboard as JSON.

Skills Applied: Prometheus, Grafana, Helm, PromQL, Monitoring

Success Outcome: *A real observability stack with a shareable dashboard.*

MINI PROJECT #10: Terraform AWS EC2 Deployment [Best done on: Day 5]

Project Brief: Write Terraform configs to provision: a VPC, public subnet, security group (port 22, 80), and an EC2 instance with a user_data script that installs Nginx. Use variables.tf and outputs.tf. Run plan, apply, verify in browser, destroy.

Skills Applied: Terraform, AWS, VPC, EC2, HCL

Success Outcome: *Infrastructure-as-code for a real cloud deployment.*

MINI PROJECT #11: Static Website CI/CD to S3 [Best done on: Day 3-5]

Project Brief: Build a personal portfolio static site (HTML/CSS). Set up a GitHub Actions pipeline that on every push to main: validates HTML, builds assets, and syncs to an AWS S3 bucket configured for static hosting.

Skills Applied: GitHub Actions, AWS S3, HTML, CI/CD

Success Outcome: *A live website updated automatically every time you push code.*

MINI PROJECT #12: Multi-Environment Deployment Pipeline [Best done on: Day 3-5]

Project Brief: Create a GitHub Actions pipeline with three environments: dev, staging, prod. Push to dev auto-deploys to a Docker container. PR to main deploys to staging. Manual approval triggers prod deployment.

Skills Applied: GitHub Actions, Environments, Approvals, Docker

Success Outcome: *A professional deployment pipeline with environment promotion gates.*

MINI PROJECT #13: Log Aggregation with ELK Lite [Best done on: Day 4-5]

Project Brief: Deploy Elasticsearch and Kibana using Docker Compose. Configure a sample app to write structured JSON logs. Create Kibana visualizations: log volume timeline, error rate chart, top endpoints dashboard.

Skills Applied: ELK Stack, Docker Compose, JSON Logging, Kibana

Success Outcome: *A searchable log analysis platform — core DevOps operations skill.*

MINI PROJECT #14: Secrets Management with Vault/AWS SSM [Best done on: Day 5]

Project Brief: Instead of hardcoding secrets in Dockerfiles or YAML, use AWS SSM Parameter Store (or HashiCorp Vault) to store DB passwords and API keys. Update your GitHub Actions pipeline to fetch secrets at runtime.

Skills Applied: Security, AWS SSM, GitHub Secrets, DevSecOps

Success Outcome: *A secure pipeline with zero hardcoded credentials — industry best practice.*

MINI PROJECT #15: Rollback Strategy Implementation [Best done on: Day 5]

Project Brief: In a Kubernetes deployment, implement a blue-green deployment strategy. Deploy v1 of an app. Deploy v2. Use Kubernetes services to switch traffic. Demonstrate instant rollback from v2 to v1 with zero downtime.

Skills Applied: Kubernetes, Blue-Green Deployment, Zero Downtime, Strategy

Success Outcome: *Proven ability to deploy safely and roll back instantly — a key SRE skill.*

10 CAPSTONE PROJECTS — Present on Day 6

Capstone projects integrate ALL week's learnings. Teams of 3-4 students. Evaluated on Day 6 using the rubric above. Every project must include: a Git repo, CI/CD pipeline, Docker containers, and a working live demo.

CAPSTONE #C01: DevOps Pipeline for an E-Commerce Microservice

What to Build: Build a mini e-commerce system with 3 microservices: product catalog API, shopping cart API, and a frontend. Each service lives in its own Docker container. A GitHub Actions pipeline automatically tests, builds, and deploys all three to Kubernetes. Prometheus monitors all services. Failing a unit test blocks deployment.

Tools & Tech Stack: Python/Node.js, Docker, Kubernetes, GitHub Actions, Prometheus, Grafana

Real-World Story: *Used by: Flipkart, Amazon, Myntra — this IS how real e-commerce runs.*

CAPSTONE #C02: Automated Cloud Infrastructure for a Startup

What to Build: Use Terraform to provision a complete AWS environment from scratch: VPC, public/private subnets, NAT gateway, EC2 app server, RDS database, S3 for assets, and CloudFront CDN. Everything in code. Running terraform apply should recreate the entire startup's cloud in under 10 minutes.

Tools & Tech Stack: Terraform, AWS (EC2, RDS, S3, VPC, CloudFront), Bash

Real-World Story: *This is what a Cloud/DevOps Engineer builds on their first week at a startup.*

CAPSTONE #C03: Zero-Downtime Deployment System

What to Build: Implement a complete blue-green deployment system on Kubernetes. Application has v1 and v2. A deployment pipeline promotes v2 to canary (10% traffic) → staging → production with automatic rollback if error rate exceeds 5% (detected via Prometheus alerts).

Tools & Tech Stack: Kubernetes, Prometheus, GitHub Actions, Docker, YAML

Real-World Story: *Netflix, Google, Meta — all use variants of this. Zero downtime = zero angry users.*

CAPSTONE #C04: CI/CD Platform for a College Codebase

What to Build: Set up a complete DevOps platform for a college department: self-hosted Gitea (or GitHub), Jenkins or GitHub Actions runner, SonarQube for code quality, Nexus for artifact storage, and automated deployment to staging. Document it as a runbook.

Tools & Tech Stack: Jenkins/GitHub Actions, SonarQube, Docker, Gitea, Nexus

Real-World Story: *Built for VIT's own use — students can demo to their HOD and propose adoption.*

CAPSTONE #C05: Real-Time IoT Data Pipeline on Cloud

What to Build: Simulate IoT sensors (temperature, humidity — using a Python script). Send data to AWS IoT Core → Kinesis Data Streams → Lambda for processing → DynamoDB storage → API Gateway → React dashboard. Full serverless DevOps pipeline with IaC deployment.

Tools & Tech Stack: AWS IoT, Lambda, Kinesis, DynamoDB, Terraform, Python, React

Real-World Story: *Connects IoT + DevOps — directly applicable to L&T's own manufacturing plants.*

CAPSTONE #C06: DevSecOps Pipeline with Automated Security Scanning

What to Build: Build a CI/CD pipeline that includes: Trivy for Docker image vulnerability scanning, Snyk for dependency scanning, OWASP ZAP for dynamic security testing, and Bandit for Python static analysis. Pipeline fails if any HIGH severity vulnerabilities found.

Tools & Tech Stack: GitHub Actions, Trivy, Snyk, OWASP ZAP, Docker, Python

Real-World Story: *DevSecOps is the fastest-growing DevOps specialization. Security that's automatic, not an afterthought.*

CAPSTONE #C07: Multi-Cloud Deployment with Failover

What to Build: Deploy the same application to both AWS and Azure using Terraform. Set up health checks. If AWS goes down (simulated by scaling EC2 to 0), traffic automatically routes to Azure via DNS failover. Demonstrate the failover live during Day 6 demo.

Tools & Tech Stack: Terraform, AWS, Azure, Route53, Docker, Kubernetes

Real-World Story: *Used by: HDFC Bank, TCS — any organization that cannot afford downtime.*

CAPSTONE #C08: Full Observability Stack for a Production App

What to Build: Deploy a Node.js application with full observability: structured logging to Elasticsearch, metrics to Prometheus, traces with Jaeger (OpenTelemetry), and a unified Grafana dashboard. Alert via Grafana when error rate > 1% or latency > 500ms.

Tools & Tech Stack: Node.js, ELK Stack, Prometheus, Jaeger, Grafana, Docker Compose

Real-World Story: *The SRE's most essential toolkit — 'You can't fix what you can't see.'*

CAPSTONE #C09: GitOps Deployment with ArgoCD

What to Build: Implement GitOps: application manifests live in a Git repo. ArgoCD watches the repo and automatically deploys changes to Kubernetes. Any drift from the desired state (someone manually changes a pod) is auto-corrected. Show the self-healing GitOps loop live.

Tools & Tech Stack: ArgoCD, Kubernetes, Git, Docker, YAML

Real-World Story: *GitOps is the cutting edge of DevOps — used by Spotify, Shopify, and most cloud-native companies.*

CAPSTONE #C10: DevOps Maturity Assessment & Implementation Plan

What to Build: Choose a real organization (college, local business, NGO). Assess their current software delivery maturity. Implement a minimum DevOps stack for them: Git repo, basic CI, containerized app, and monitoring. Document before/after. Present ROI (time saved, errors reduced).

Tools & Tech Stack: GitHub, Docker, GitHub Actions, Prometheus/UptimeRobot, Docs

Real-World Story: *This is consulting work. This is what senior DevOps engineers do. If students can present this, they're employable.*